

Informe PC4 - Desarrollo de Software

Curso: CC3S2 Desarrollo de Software **Alumna:** Melissa Iman Noriega **Fecha:** 01/07/2026 **Repositorio:** <https://github.com/Melissa1221/pc4-desarrollo-software>

1. Descripción

Se desarrolló el núcleo de dominio en Java (Maven) de una plataforma de rescate y adopción de mascotas. El trabajo cubre los tres módulos del enunciado y se centra en los patrones de diseño de comportamiento, que fue lo que se pidió priorizar. Según la indicación, no se implementaron las búsquedas por especialidad: solo se desarrolló la interfaz de cada módulo siguiendo los patrones. Los datos se manejan en memoria.

2. Requisitos cumplidos

Módulo 1 — Reporte de animales perdidos y alertas

- RF 1.1: se registra una mascota perdida con nombre, especie, raza, foto y descripción (LostPet).
- RF 1.2: el reporte guarda las coordenadas geográficas.
- RF 1.3: un ciudadano anónimo registra un avistamiento con foto y ubicación (Sighting).
- RF 1.4: al reportar una mascota se notifica a los suscriptores dentro de un radio configurable (AlertCenter).
- RNF 1.2 (anonimato): el id del dueño no se expone en la alerta que reciben los ciudadanos.

Módulo 2 — Buscador por imagen

- RF 2.1: se carga una imagen para la búsqueda (ImageQuery).
- RF 2.2: la búsqueda se resuelve según la intención elegida (adopción, venta, verificar pérdida).
- RF 2.3 / 2.4 / 2.5: cada intención tiene su propia clase; la búsqueda concreta se deja como interfaz, sin implementar el motor.
- RNF 2.1 (abstracción): los metadatos van en un mapa clave-valor, de modo que el motor es intercambiable.

Módulo 3 — Red de cuidadores

- RF 3.1: el perfil guarda el rol del cuidador (solidario, profesional, especializado).
- RF 3.2: el cuidador define sus restricciones (especies aceptadas).
- RF 3.3: interruptor para activar/desactivar las alertas del módulo 1.
- RNF 3.1 (confiabilidad): el perfil solo se habilita si pasa la validación del documento.

3. Patrones de diseño de comportamiento usados

Patrón	Dónde	Para qué
Observer	AlertObserver, AlertCenter	notificar a los cuidadores suscritos en un radio (RF 1.4)
State	ReportState y sus estados	el reporte cambia de comportamiento según su estado (activo, encontrado, cerrado)
Strategy	SearchStrategy	cada intención de búsqueda es una estrategia intercambiable
Template Method	SearchTemplate	fija el flujo de la búsqueda (validar, buscar, formatear)

Patrón	Dónde	Para qué
Command	AlertCommand, AlertSwitch	activar/desactivar alertas como acciones (RF 3.3)
Chain of Responsibility	ProfileValidator y sus validadores	validar el perfil paso a paso antes de habilitarlo (RNF 3.1)

Los patrones se tomaron como referencia del repositorio visto en clase (peterm85/design-patterns), respetando su estructura.

4. Arquitectura

El código está separado por módulo en tres paquetes independientes:

com.uni.petfinder

```
├─ alertas      → módulo 1 (Observer + State)
├─ busqueda    → módulo 2 (Strategy + Template Method)
├─ cuidadores  → módulo 3 (Command + Chain of Responsibility)
└─ App         → demostración de los tres módulos
```

La separación en paquetes responde al RNF 3.2 (escalabilidad): los módulos no dependen entre sí, así que un pico de demanda en uno no afecta al otro. La comunicación es por interfaces, lo que permite reemplazar implementaciones sin tocar el resto.

5. Cómo ejecutar

```
mvn compile
mvn exec:java
```

La clase App ejecuta los tres módulos e imprime el resultado de cada patrón.